

CO5- LAB EXPERIMENTS

1. Simple Chatbot using NLP

Code

```
def chatbot(user_input):  
    user_input = user_input.lower()  
  
    if "hello" in user_input or "hi" in user_input:  
        return "Hello! How can I help you?"  
    elif "name" in user_input:  
        return "I am a simple NLP chatbot."  
    elif "how are you" in user_input:  
        return "I am fine. Thank you!"  
    elif "bye" in user_input:  
        return "Goodbye!"  
    else:  
        return "Sorry, I don't understand."  
  
while True:  
    user = input("You: ")  
    response = chatbot(user)  
    print("Bot:", response)  
  
    if "bye" in user.lower():  
        break
```

Input

You: Hello

You: What is your name?

You: Bye

Output

Bot: Hello! How can I help you?

Bot: I am a simple NLP chatbot.

Bot: Goodbye!

2. K-Means Clustering

Code

```
from sklearn.cluster import KMeans
```

```
X = [
```

```
    [1, 2],
```

```
    [1, 3],
```

```
    [2, 2],
```

```
    [8, 8],
```

```
    [9, 8],
```

```
    [8, 9]
```

```
]
```

```
model = KMeans(n_clusters=2, random_state=0, n_init=10)
```

```
model.fit(X)
```

```
print("Cluster Labels:", model.labels_)
```

```
print("Cluster Centers:")
```

```
print(model.cluster_centers_)
```

Input

No input required.

Output

Cluster Labels: [1 1 1 0 0 0]

Cluster Centers:

```
[[8.33333333 8.33333333]  
[1.33333333 2.33333333]]
```

3. Reinforcement Learning Agent

Code

```
import random  
  
q_table = [0, 0, 0, 0, 0]  
  
for episode in range(100):  
    state = 0  
  
    while state != 4:  
        action = random.choice([-1, 1])  
        new_state = state + action  
  
        if new_state < 0 or new_state > 4:  
            new_state = state  
  
        reward = 10 if new_state == 4 else -1  
  
        q_table[new_state] += 0.1 * (  
            reward + 0.9 * max(q_table) - q_table[new_state]  
        )  
  
        state = new_state  
  
print("Q-Values:")  
print(q_table)
```

```
print("Goal State: 4")
```

Input

No input required.

Output

Q-Values:

```
[-10.23, -9.12, -7.45, 1.85, 9.99]
```

Goal State: 4

4. Image Segmentation using Computer Vision

Code

```
import cv2
```

```
image = cv2.imread("image.jpg")
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
_, segmented = cv2.threshold(  
    gray, 127, 255, cv2.THRESH_BINARY  
)
```

```
cv2.imwrite("segmented.jpg", segmented)
```

```
print("Image segmentation completed.")
```

```
print("Segmented image saved as segmented.jpg")
```

Input

Input image: image.jpg

Output

Image segmentation completed.

Segmented image saved as segmented.jpg

5. Markov Chain Model

Code

```
import random

transitions = {
    "Sunny": ["Sunny", "Rainy"],
    "Rainy": ["Sunny", "Rainy"]
}

state = "Sunny"

print("Starting State:", state)

for i in range(5):
    state = random.choice(transitions[state])
    print("Next State:", state)
```

Input

No input required.

Starting State: Sunny

Output

Starting State: Sunny

Next State: Rainy

Next State: Sunny

Next State: Sunny

Next State: Rainy

Next State: Rainy

6. Object Detection using Pre-trained Model

Code

```
import cv2
```

```
image = cv2.imread("image.jpg")
```

```
net = cv2.dnn.readNetFromCaffe(  
    "MobileNetSSD_deploy.prototxt",  
    "MobileNetSSD_deploy.caffemodel"  
)
```

```
blob = cv2.dnn.blobFromImage(  
    cv2.resize(image, (300, 300)),  
    0.007843,  
    (300, 300),  
    127.5  
)
```

```
net.setInput(blob)
```

```
detections = net.forward()
```

```
classes = [  
    "background", "aeroplane", "bicycle", "bird",  
    "boat", "bottle", "bus", "car", "cat", "chair",  
    "cow", "diningtable", "dog", "horse", "motorbike",  
    "person", "pottedplant", "sheep", "sofa",  
    "train", "tvmonitor"  
]
```

```
for i in range(detections.shape[2]):  
    confidence = detections[0, 0, i, 2]  
  
    if confidence > 0.5:  
        class_id = int(detections[0, 0, i, 1])  
        print("Detected:", classes[class_id])
```

```
print("Object detection completed.")
```

Input

Input image: image.jpg

Pre-trained model files:

MobileNetSSD_deploy.prototxt

MobileNetSSD_deploy.caffemodel

Output

Detected: person

Detected: car

Object detection completed.

7. Hyperparameter Tuning

Code

```
from sklearn.datasets import load_iris  
from sklearn.model_selection import GridSearchCV  
from sklearn.svm import SVC
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
model = SVC()
```

```
parameters = {  
    "C": [0.1, 1, 10],  
    "kernel": ["linear", "rbf"]  
}
```

```
grid = GridSearchCV(  
    model,  
    parameters,  
    cv=5  
)
```

```
grid.fit(X, y)
```

```
print("Best Parameters:", grid.best_params_)  
print("Best Accuracy:", round(grid.best_score_, 2))
```

Input

No input required.

Iris dataset is automatically loaded.

Output

Best Parameters: {'C': 1, 'kernel': 'linear'}

Best Accuracy: 0.98

8. Bayesian Network

Code

```
from pgmpy.models import DiscreteBayesianNetwork
```

```
from pgmpy.factors.discrete import TabularCPD
```

```
from pgmpy.inference import VariableElimination
```



```
model = DiscreteBayesianNetwork([  
    ("Rain", "WetGrass")  
])
```

```
cpd_rain = TabularCPD(  
    variable="Rain",  
    variable_card=2,  
    values=[[0.7], [0.3]]  
)
```

```
cpd_wet = TabularCPD(  
    variable="WetGrass",  
    variable_card=2,  
    values=[  
        [0.9, 0.2],  
        [0.1, 0.8]  
    ],  
    evidence=["Rain"],  
    evidence_card=[2]  
)
```

```
model.add_cpds(cpd_rain, cpd_wet)
```

```
model.check_model()
```

```
inference = VariableElimination(model)
```

```
result = inference.query(  

```

```
variables=["WetGrass"],  
evidence={"Rain": 1}  
)
```

```
print(result)
```

Input

Rain = True

Output

```
+-----+-----+  
| WetGrass | phi(WetGrass) |  
+=====+=====+  
| 0        | 0.2000    |  
+-----+-----+  
| 1        | 0.8000    |  
+-----+-----+
```

9. Face Recognition using Pre-trained Model

Code

```
import face_recognition
```

```
known_image = face_recognition.load_image_file("known.jpg")
```

```
known_encoding = face_recognition.face_encodings(known_image)[0]
```

```
unknown_image = face_recognition.load_image_file("unknown.jpg")
```

```
unknown_encoding = face_recognition.face_encodings(unknown_image)[0]
```

```
result = face_recognition.compare_faces(  
    [known_encoding],  
    unknown_encoding
```

)

```
if result[0]:
```

```
    print("Face Recognized")
```

```
else:
```

```
    print("Face Not Recognized")
```

Input

Known image: known.jpg

Test image: unknown.jpg

Output

Face Recognized

10. Genetic Algorithm to Evolve a Neural Network

Code

```
import random
```

```
def fitness(weights):
```

```
    return -abs(sum(weights) - 10)
```

```
population = [
```

```
    [random.uniform(0, 5) for _ in range(3)]
```

```
    for _ in range(10)
```

```
]
```

```
for generation in range(20):
```

```
    population.sort(key=fitness, reverse=True)
```

```
    parents = population[:2]
```

```
new_population = parents[:]
```

```
while len(new_population) < 10:
```

```
    child = [  
        (parents[0][i] + parents[1][i]) / 2  
        for i in range(3)  
    ]
```

```
    index = random.randint(0, 2)
```

```
    child[index] += random.uniform(-0.5, 0.5)
```

```
    new_population.append(child)
```

```
population = new_population
```

```
best = max(population, key=fitness)
```

```
print("Best Neural Network Weights:")
```

```
print([round(x, 2) for x in best])
```

```
print("Fitness:", round(fitness(best), 2))
```

Input

No input required.

Output

Best Neural Network Weights:

[3.21, 3.45, 3.34]

Fitness: -0.00

